Laboratory Manual

Of

Engineering Mathematics-II with Computing Tools

(2ET1000102P)

B.Tech. First year /Semester-II

(CE/IT/ICT/AI&DS/CIVIL/ME/EE)

2025-26

Faculty of Engineering & Technology

Sankalchand Patel University

Sankalchand Patel College of Engineering,
Visnagar, Gujarat.

Certificate

This is to certify that the Lab/Term work carried out in the subject Engineering Mathematics-II with Computing Tools (2ET1000102P) and recorded in this journal is the bonafide work of Mr./Ms./Mrs._____, PRN No. _____of B.Tech. Semester _____in the branch of CE/IT/ICT/AI&DS/CIVIL/ME/EE of Faculty of Engineering and Technology during the academic year 2025-2026.

Faculty in Charge **Head of the Department**

Date: **Date:**

Engineering Mathematics-II with Computing Tools(2ET1000102P)

Practical list

1.	Create vectors in Scilab.
2.	Operations on Vectors in Scilab.
3.	Find sum, mean, length, max, min, prod, sign, find, fix, flour, ceil, round, sort using inbuilt commands. (Single line Commands)
4.	Creating Matrices in Scilab.
5.	Some Special Matrices through in-built functions.
6.	Perform different basic operations of matrix (single line Commands).
7.	Basic operation matrix addition and subtraction using script file. (Multiple Commands)
8.	Basic operation matrix multiplication using script file.
9.	Write a SCILAB program to draw the graph of following functions for a given data (1) $y = \sin x,\ x \in [0,2\pi]$ (2) $y = \cos x,\ x \in [-\pi, \pi]$ (3) $a = 10e^{-0.2t}, t \in [0, 50]$ (4) $y = 3x + 5, x \in [0,10]$ (5) $y = x^3 + 2x^2 - 5, x \in [-10,10]$ (6) $a = [1\ \ 13\ \ 6\ \ 9\ \ 23\ \ 12]$
10.	Write a SCILAB program to draw a multiple graph using style operation, hold command, line command.
11.	Write a SCILAB program to draw a graph using subplot command.
12	Write a SCILAB program to draw Simple 2-D graphs.
13	Write a SCILAB program to draw the 3-D plot such as plot-3, bar-3, mesh, surf, etc.
14.	Write a SCILAB program to check linear independence of vectors using conditional statement.
15.	Write a SCILAB program to check orthogonality of vectors by inner product.

Scilab works with essentially only one kind of object a rectangular, numerical array of numbers, possibly complex, called a matrix. In some situations, 1×1 matrices are interpreted as scalars and matrices with only one row or one column are interpreted as vectors. Matrices can be introduced into Scilab in several different ways:

- Entered by an explicit list of elements.
- Generated by built-in statements and functions.
- Created in Script files.
- Loaded from external data files.
- Scilab contains no size or type declarations for variables. Scilab allocates storage
- Automatically, up to available memory.

<u>Practical 1</u>: Create vectors in Scilab.

Vectors come in two formats - row vectors and column vectors. In either case they are lists of numbers separated by either commas or spaces.

- The number of entries is known as the "length" of the vector and the entries· are called elements or components of the vector.
- The entries must be enclosed by square brackets "[" and "]". Entries of a row vector are separated by comma (,) or space, whereas entries of a column vector are separated by semicolon (;).
- A row vector can be transposed to a column vector and vice-versa using the transpose operator (.'). Whereas, transpose conjugate operation be performed by using ('). Thus, both of these operators give the output for real vectors.
- Binary operations applicable on vectors are addition (+), subtraction (-), scalar multiplication (*), and dot product (.*).
- A row (column) vector can be joined with another row (column) vector. The same is demonstrated below.

Following Scilab session illustrates features of Scilab vectors discussed above.

```
-->  v = [1, 3, sqrt(5)]

v  =

   1.   3.   2.236068

--> length(v)

ans  =

   3.

-->  v2 = 3*v

 v2  =

   3.   9.   6.7082039

--> v + v2

ans  =

   4.   12.   8.9442719

--> v - v2

ans  =

  -2.  -6.  -4.472136

--> v3 = v.'

 v3  =

   1.
```

```
   3.

   2.2360680

--> v4 = v2.'

 v4  =

   3.

   9.

   6.7082039

--> v5 = [v, v2]

 v5  =

   1.   3.   2.236068   3.   9.   6.7082039

--> v6 = [v3; v4]

 v6  =

   1.

   3.

   2.2360680

   3.

   9.

   6.7082039

--> v6 = [v3, v4]

 v6  =

   1.        3.

   3.        9.

   2.236068   6.7082039
```

Remark: 1. It is to be noted from the above demonstration of Scilab session that the dimensions of vectors must agree for joining the vectors. 2. In all vector arithmetic with vectors of equal length, the operations are carried out element-wise.

Remark: 2. In all vector arithmetic with vectors of equal length, the operations are carried out element-wise.

Particular entries of a vector can be accessed and changed as demonstrated below.

```
--> v
v  =
   1.   3.   2.236068
-->v(1)
ans  =
   1.
-->v(2)
ans  =
   3.
-->v(3)
ans  =
   2.2360680
-->v(2) = 5
v  =
   1.   5.   2.236068
```

<u>Practical 2</u>: Operations on Vectors in Scilab.

In Scilab, operations are available for addition, subtraction, vector product, element-wise product, element-wise division, and element-wise power.

Addition (+) and subtraction (-) of vectors:

The addition (+) and subtraction (-) operations in Scilab work element-wise by giving output as addition and subtraction of corresponding elements.
For example:

```
--> a = [5 7 9; 1 -3 -7], b = [-1 2 5; 9 0 5]

a  =

   5.   7.   9.

   1.  -3.  -7.

b  =

  -1.   2.   5.

   9.   0.   5.

-->a+b

ans  =

   4.    9.   14.

   10.  -3.  -2.

--> a-b

ans  =

   6.   5.   4.

  -8.  -3.  -12.
```

Vector product (*):

We shall describe two ways in which a meaning may be attributed to the product of two vectors. In both cases, the vectors concerned must have the same length. The first product is the standard scalar product. Suppose that u and v are two vectors of length n, u being a row vector and v a column vector:
$u = [u_1, u_2, \ldots\ldots, u_n]$, $v = [v_1 ; v_2; \ldots\ldots; v_n]$

The scalar product is defined by multiplying the corresponding elements together and adding the results to give a single number (scalar).

$$u * v = \sum_{i=1}^{n} u_i v_i$$

We can perform this product in Scilab by

```
-->u = [10, -11, 12], v = [20; -21; -22]

 u =

   10.  -11.   12.

v  =

   20.

  -21.

  -22.

--> prod=u*v                          // row times column vector

prod  =

   167.
```

Suppose we also define a row vector w and a column vector z by

```
-->  w=[2,1,3], z=[7;6;5]

w  =

   2.   1.   3.

z  =

   7.

   6.

   5.
```

and we want to form the scalar products of u with w and v with z.

```
--> u*w

Inconsistent row/column dimensions.
```

An error results because w is not a column vector. Recall from earlier that transposing (with ') turns column vectors into row vectors and vice versa. So, to form the scalar product of two row vectors or two column vectors,

```
-->  u*w.'                    // u and w are row vectors

ans  =

   45.

--> u*u.'                     // u is a row vector

ans  =

   365.

--> v.'*z                     // v and z are column vectors

ans  =

  -96.
```

We shall refer to the Euclidean length of a vector as the norm of a vector; it is denoted by the symbol ‖. ‖and defined by

$$\|u\| = \sqrt{\sum_{i=1}^{n} u_i^2}$$

where n is its dimension. This can be computed in Scilab in one of two ways exemplified below:

```
--> [sqrt(u*u.'), norm(u)]

ans =

  19.104973   19.10497
```

Where norm is a built in Scilab function that accepts a vector u as input and delivers a scalar ‖u‖ as the output.

Element-wise product or dot product (.*)

The second way of forming the product of two vectors of the same length is known as the Hadamard product. It is not often used in Mathematics but is an invaluable Scilab feature. It involves vectors of the same type. If u and v are two vectors of the same type (both row vectors or both column vectors), the mathematical definition of this product, which we shall call the dot product, is the vector having the components

$$u.v = [u_1v_1, u_2v_2, \ldots\ldots, u_nv_n]$$

The result is a vector of the same length and type as u and v. Thus, we simply multiply the corresponding elements of two vectors. In Scilab, the product is computed with the operator.* and, using the vectors w, z defined earlier

```
--> w.*w

ans =

  4.  1.  9.

--> w.*z'

ans =

  14.  6.  15.
```

Element-wise division of vectors: right-division (./) and left-division (.\)

There is no mathematical definition for the division of one vector by another. However, in Scilab, operators "./" and ".\" is defined to give element-wise division. It is therefore only defined for vectors of the same size and type. The right-division opertor"./" divides elements of pre-factor with corresponding elements of post-factor. Whereas, left-division operator ".\" divides elements of post-factor with corresponding elements of pre-factor. That is, for vectors a and b of same shape and length, a./b gives the vector with entries a(i)/b(i) for each index i and a.\b gives the vector with entries b(i)/a(i) for each index i.

```
--> a = 1:5, b=6:10, a./b, a.\b
a =

  1.  2.  3.  4.  5.
b =
```

```
  6.  7.  8.  9.  10.
ans  =

  0.1666667  0.2857143  0.375  0.4444444  0.5
ans  =

  6.  3.5  2.6666667  2.25  2.
-->  a./a

ans  =

  1.  1.  1.  1.  1.
-->  c = -2:2, a./b

c  =

 -2. -1.  0.  1.  2.
ans  =

  0.1666667  0.2857143  0.375  0.4444444  0.5
```

Remark: The importance of these operations will be highlighted during their discussion on two dimensional matrices as same operations are applicable there also.

Element-wise power of vectors (.^)

To square each of the elements of a vector we could, for example, do u.*u. However, a neater way is to use the .^ operator is demonstrated below.

```
--> u
u  =

  10. -11.  12.
--> u.^2
ans  =

  100.  121.  144.
--> u.*u
ans  =

  100.  121.  144.
-->  u.*w.^(-2)
ans  =
2.5  -11.  1.3333333
```

Observe that powers (.^ in this case) are done first, before any other arithmetic operation.

<u>Practical 3</u>: If $x = [1411100], y = [14\ 200\ \ -100\ ]$ **and** $z = [1.410.7 - 1.120.9]$ **then find sum, mean, length, max, min, prod, sign, find, fix, flour, ceil, round, sort using inbuilt commands. (Single line Commands)**

	--> x = [1 4 11 100] x = 1. 4. 11. 100.	--> y = [14;200;-100] y = 14. 200. -100.	--> z = [1.4 10.7 -1.1 20.9] z = 1.4 10.7 -1.1 20.9
1.	--> sum(x) ans = 116.	--> sum(y) ans = 114.	--> sum(z) ans = 31.9
2.	--> mean(x) ans = 29.	--> mean(y) ans = 38.	--> mean(z) ans = 7.975
3.	--> length(x) ans = 4.	--> length(y) ans = 3.	--> length(z) ans = 4.
4.	--> max(x) ans = 100.	--> max(y) ans = 200.	--> max(z) ans = 20.9
5.	--> min(x) ans = 1.	--> min(y) ans = -100.	--> min(z) ans = -1.1
6.	--> prod(x) ans = 4400.	--> prod(y) ans = -280000.	--> prod(z) ans = -344.3902
7.	--> fix(x) ans = 1. 4. 11. 100.	--> fix(y) ans = 14. 200. -100.	--> fix(z) ans = 1. 10. -1. 20.
8.	--> floor(x) ans = 1. 4. 11. 100.	--> floor(y) ans = 14. 200. -100.	--> floor(z) ans = 1. 10. -2. 20.
9.	--> ceil(x) ans = 1. 4. 11. 100.	--> ceil(y) ans = 14. 200. -100.	--> ceil(z) ans = 2. 11. -1. 21.
10.	--> round(x) ans = 1. 4. 11. 100.	--> round(y) ans = 14. 200. -100.	--> round(z) ans = 1. 11. -1. 21.

The Colon Operator and Creating linearly spaced vector: linspace function in Scilab.

Scilab has an in-built function linspace for creating a linearly spaced vector with any specified values between two real or complex numbers. The syntax for calling this function is

[v] = linspace(x1, x2, n)

Here, x1 and x2 are real or complex scalars or column vectors and n is a natural number whose value should be greater than or equal to 2.
For example:

```
-->eye(2, 2)

ans  =

   1.   0.
   0.   1.
-->eye(4, 4)

ans  =

   1.   0.   0.   0.
   0.   1.   0.   0.
   0.   0.   1.   0.
   0.   0.   0.   1.

-->linspace(1,2,5)

ans  =

   1.   1.25   1.5   1.75   2.

-->linspace(1,10,10)

ans  =

   1.   2.   3.   4.   5.   6.   7.   8.   9.   10.

-->linspace(1+%i,2+2*%i,5)

ans  =

   1. + i     1.25 + 1.25i   1.5 + 1.5i   1.75 + 1.75i   2. + 2.i

-->linspace([1:4]',[5:8]',5)

ans  =

   1.   2.   3.   4.   5.
   2.   3.   4.   5.   6.
   3.   4.   5.   6.   7.
   4.   5.   6.   7.   8.
```

<u>Practical 4</u>: Creating Matrices in Scilab.

The following is the list of symbols used to define a matrix:
· square brackets "[" and "]"mark the beginning and the end of the matrix,
· commas "," or spaces separate the values on different columns,
· semicolons ";" separate the values of different rows.

The following syntax can be used to define a matrix, where blank spaces are optional (but make the line easier to read) and "..." are designing intermediate values:

$$A = [a_{11}, a_{12}, \ldots, a_{1n};\ a_{21}, a_{22}, \ldots, a_{2n};\ \ldots;\ a_{n1}, a_{n2}, \ldots, a_{nn}]$$

For example:

--> A = [1 2 3; 4 5 6; 7 8 9]

A =

1. 2. 3.

4. 5. 6.

7. 8. 9.

And

--> A = [1 2 3

> 4 5 6

> 7 8 9]

A =

1. 2. 3.

4. 5. 6.

7. 8. 9.

Scilab always prints out the variable in the executed line (which can be very awkward and time consuming for large arrays) unless you end the line with a semicolan (;). It is good practice to use the semicolan at the end of the line. You can always ask Scilab to print the matrix later also by calling the variable name, as shown below.

--> A

A =

1. 2. 3.
4. 5. 6.
7. 8. 9.

Task to Perform:

(1)

(a) Create a matrix $A = [3\ 4\ 5\ 1\ 3\ 2\ 5\ 6\ 9\ ]$

(b) Transpose the matrix A.

(c) Create the submatrix of A by 1st and 2nd row, 2nd and 3rd column.

(d) Set elements in 3rd row and 2nd column to zero.

(e) Set elements of 1st to 3rd row and 1st column to zero.

(f) Set elements of 2nd row of 3rd column to one.

(g) Set elements in 3rows of 3rd column to eight.

(h) Delete the second Column.

	Input	Output
a.	--> A=[3 4 5;1 3 2;5 6 9]	A = 3. 4. 5. 1. 3. 2. 5. 6. 9.
b.	--> B=A'	B = 3. 1. 5. 4. 3. 6. 5. 2. 9.
c.	--> C=A(1:2,2:3)	C = 4. 5. 3. 2.
d.	-->A(3,2)=0	A = 3. 4. 5. 1. 3. 2. 5. 0. 9.
e.	-->A(1:3,1)=0	A = 0. 4. 5. 0. 3. 2. 0. 0. 9.
f.	-->A(2,3)=1	A = 0. 4. 5. 0. 3. 1. 0. 0. 9.
g.	-->A(1:3,3)=8	A = 0. 4. 8. 0. 3. 8. 0. 0. 8.
h.	--> A(:,2)=[ ]	A = 0. 8. 0. 8. 0. 8.

(2)

(a) Create the matrix $A = [2\ 4\ 6\ 5\ 7\ 6\ 1\ 8\ 4\ ]$**.**

(b) B = 1st to 3rd row and 2nd and 3rd column.

(c) C = The entire row and only 2nd column.

(d) D = 1st , 2nd row and only 2nd column.

(e) E= Only 1st row and entire column.

	Input	Output
(a)	--> A=[2 4 6;5 7 6;1 8 4]	A = 2. 4. 6. 5. 7. 6. 1. 8. 4.
(b)	--> B=A(1:3,2:3)	B = 4. 6. 7. 6. 8. 4.
(c)	--> C=A(:,2)	C = 4. 7. 8.
(d)	--> D=A(1:2,2)	D = 4. 7.
(e)	--> E=A(1,:)	E = 2. 4. 6.

(f) Reshape the matrix A = $\begin{bmatrix} 1 & 0 & 2 & 4 & 1 & 4 \\ 1 & 1 & 0 & 2 & 6 & 4 \\ 2 & 9 & 2 & 3 & 1 & 5 \\ 4 & 0 & 2 & 0 & 0 & 8 \\ 2 & 1 & 3 & 7 & 6 & 8 \\ 6 & 0 & 0 & 3 & 7 & 0 \end{bmatrix}$ **into** 9×4 **matrix.**

```
--> A=[1 0 2 4 1 4; 1 1 0 2 6 4;2 9 2 3 1 5;4 0 2 0 0 8;2 1 3 7 6 8;6 0 0 3 7 0]
A  =

   1.   0.   2.   4.   1.   4.
   1.   1.   0.   2.   6.   4.
   2.   9.   2.   3.   1.   5.
   4.   0.   2.   0.   0.   8.
   2.   1.   3.   7.   6.   8.
   6.   0.   0.   3.   7.   0.

--> matrix(A,9,4)
ans  =

   1.   0.   4.   0.
   1.   1.   2.   6.
   2.   0.   3.   7.
   4.   2.   0.   4.
   2.   0.   7.   4.
   6.   2.   3.   5.
   0.   2.   1.   8.
   1.   3.   6.   8.
   9.   0.   1.   0.
```

<u>Practical 5</u>: Some Special Matrices through in-built functions.

Identity matrix: The command eye(n, n) creates the identity matrix of order n × n.

For example:

```
-->eye(2, 2)

ans  =

  1.  0.

  0.  1.

-->eye(3, 3)

ans  =

  1.  0.  0.

  0.  1.  0.

  0.  0.  1.
```

Remark: For the case of $m \neq n$, the Scilab command eye(m, n)creates a rectangular matrix with m rows and n columns with the identity matrix of order = min$\{m, n\}$ and additional rows or columns as zeros.

For example:

```
-->eye(2, 3)

ans  =


  1.  0.  0.

  0.  1.  0.

-->eye(4, 3)

ans  =

  1.  0.  0.

  0.  1.  0.

  0.  0.  1.

  0.  0.  0.
```

Matrix with all entries one

The command ones(m, n) creates the matrix of order m × n having its all elements one.

For example:

```
-->ones(2, 3)

ans  =

   1.   1.   1.

   1.   1.   1.
```

Matrix with all entries zero

The command zeros(m, n) creates the matrix of order m × n having its all element zero.

For example:

```
-->zeros(2, 3)

ans  =

   0.   0.   0.

   0.   0.   0.
```

Random matrix

The command rand(m, n) will create a matrix of order m × n with randomly generated entries distributed uniformly between 0 and 1.

```
--> rand(5, 5)

ans  =

   0.2113249   0.6283918   0.5608486   0.2320748   0.3076091

   0.7560439   0.8497452   0.6623569   0.2312237   0.9329616

   0.0002211   0.685731    0.7263507   0.2164633   0.2146008

   0.3303271   0.8782165   0.1985144   0.8833888   0.312642

   0.6653811   0.068374    0.5442573   0.6525135   0.3616361
```

While rand(m,n) will create an m × n array.

```
--> rand(3,4)

ans  =

   0.2922267   0.3321719   0.4368588   0.4051954

   0.5664249   0.5935095   0.2693125   0.9184708

   0.4826472   0.5015342   0.6325745   0.0437334
```

Task to perform:

(1) If $V = [1\ 2\ 3\ 4]$ **then**

- **(i) Form a diagonal matrix A by using the elements of V as the main diagonal elements of A.**
- **(ii) Form amatrix B by using the elements of V as the elements of first upper diagonal of B.**
- **(iii) Form amatrix C by using the elements of V as the elements of first lower diagonal of B.**

	Input	Output
1.	--> V=[1 2 3 4]	V = 1. 2. 3. 4.
2.	--> A=diag(V)	A = 1. 0. 0. 0. 0. 2. 0. 0. 0. 0. 3. 0. 0. 0. 0. 4.
3.	--> B=diag(V,1)	B = 0. 1. 0. 0. 0. 0. 0. 2. 0. 0. 0. 0. 0. 3. 0. 0. 0. 0. 0. 4. 0. 0. 0. 0. 0.
4.	--> C=diag(V,-1)	C = 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 2. 0. 0. 0. 0. 0. 3. 0. 0. 0. 0. 0. 4. 0.

(2) Let x = [2 3 4 5] **and** y = [1 2 3 4 5 6 7 8 9] **then**

(i) A column vector compared diagonal element of x.

(ii) A column vector compared diagonal element of y.

(iii)A matrix having from first upper and lower diagonal matrix of x**and** y.

	Input	Output
	--> x = [2 3;4 5]	x = 2. 3. 4. 5.
	--> y = [1 2 3;4 5 6;7 8 9]	y = 1. 2. 3. 4. 5. 6. 7. 8. 9.
i.	-->diag(x)	ans = 2. 5.
ii.	-->diag(y)	ans = 1. 5. 9.
iii.	-->diag(x,1) -->diag(x,-1) -->diag(y,1) -->diag(y,-1)	ans = 3. ans = 4. ans = 2. 6. ans = 4. 8.

<u>Practical 6</u>: Perform different basic operation of matrix (single line Commands).

(1) If A= [1 2 3 0 4 5 9 2 3] **then find det(A), rank(A), trace(A), inv(A), norm(A), spec(A) and A' .**

	Input	Output
	--> A=[1 2 3;0 4 5;9 2 3]	A = 1. 2. 3. 0. 4. 5. 9. 2. 3.
i.	--> det(A) // To find Determinant	ans = -16.000000
ii.	--> rank(A) // To find Rank	ans = 3.
iii.	--> trace(A) // TO find Trace	ans = 8.
iv.	--> inv(A) // To find Inverse	ans = -0.125 0. 0.125 -2.8125 1.5 0.3125 2.25 -1. -0.25
v.	--> norm(A)	ans = 10.547504
vi.	--> spec(A) // To find eigen values	ans = -2.3811735 + 0.i 0.6936083 + 0.i 9.6875652 + 0.i
vii.	--> A' // To find Transpose	ans = 1. 0. 9. 2. 4. 2. 3. 5. 3.

(2) Let the matrix A is given by A=[2 6 3 4] and B=[1 7 − 2 8]then show that the use of arithmetic operators on these matrices. (Single line command)

	Input	Output
1.	--> A=[2 6;3 4]	A = 2. 6. 3. 4.
2.	--> B=[1 7;-2 8]	B = 1. 7. -2. 8.
3.	--> A+B	ans = 3. 13. 1. 12.
4.	--> A-B	ans = 1. -1. 5. -4.
5.	--> A*B	ans = -10. 62. -5. 53.
6.	--> A.*B	ans = 2. 42. -6. 32.
7.	--> A/B	ans = 1.2727273 -0.3636364 1.4545455 -0.7727273
8.	--> A./B	ans = 2. 0.8571429 -1.5 0.5
9.	--> A\B	ans = -1.6 2. 0.7 0.5
10.	--> A.\B	ans = 0.5 1.1666667 -0.6666667 2.
11.	--> A^2	ans = 22. 36. 18. 34.
12.	--> A.^2	ans = 4. 36. 9. 16.
13.	--> A'	ans = 2. 3. 6. 4.
14.	--> A.'	ans = 2. 3. 6. 4.

<u>Practical 7</u>: Basic operation matrix addition and subtraction using script file.

(Multiple Commands)

```
// Matrix Addition and Subtraction script file
clc
m=input("enter number of rows of the Matrix: ");
n=input("enter number of columns of the Matrix: ");
disp('enter the first Matrix')
fori=1:m
forj=1:n
A(i,j)=input('\');
end
end
disp('enter the second Matrix')
fori=1:m
forj=1:n
B(i,j)=input('\');
end
end
fori=1:m
forj=1:n
C(i,j)=A(i,j)+B(i,j);
D(i,j)=A(i,j)-B(i,j);
end
end
disp('The first matrix is')
disp(A)
disp('The Second matrix is')
disp(B)
disp('The sum of the two matrices is')
disp(C)
disp('The subtraction of the two matrices is')
disp(D)
```

```
/*Output:
Enter number of rows of the Matrix: 2
Enter number of columns of the Matrix: 2

"enter the first Matrix"
\1

\2

\3

\4
```

```
"enter the second Matrix"
\2

\4

\6

\8

"The first matrix is"
1.   2.
3.   4.

"The Second matrix is"
2.   4.
6.   8.
"The sum of the two matrices is"
3.   6.
9.  12.

"The subtraction of the two matrices is"
-1.  -2.
-3.  -4.
```

<u>Practical 8</u>: Basic operation matrix multiplication using script file.

```
// matrix multiplication script file
clc
m=input("Enter number of rows of the first Matrix: ");
n=input("Enter number of columns of the first Matrix: ");
p=input("Enter number of rows of the second Matrix: ");
q=input("Enter number of columns of the second Matrix: ");
if n==p
disp('Matrices are conformable for multiplication')
else
disp('Matrices are not conformable for multiplication')
break;
end
disp('enter the first Matrix')
for i=1:m
for j=1:n
A(i,j)=input('\');
end
end
disp('enter the second Matrix')
for i=1:p
for j=1:q
B(i,j)=input('\');
end
end
C=zeros(m,q);
for i=1:m
for j=1:q
for k=1:n
C(i,j)=C(i,j)+A(i,k)*B(k,j);
end
end
end
disp('The first matrix is')
disp(A)
disp('The Second matrix is')
disp(B)
disp('The product of the two matrices is')
disp(C)
```

```
/*output:
Enter number of rows of the first Matrix: 2
Enter number of columns of the first Matrix: 3
Enter number of rows of the second Matrix: 3
Enter number of columns of the second Matrix: 2
```

```
"Matrices are conformable for multiplication"
"enter the first Matrix"
\1
\2
\3
\4
\5
\6

"enter the second Matrix"
\1
\6
\2
\4
\3
\9

"The first matrix is"
 1.  2.  3.
 4.  5.  6.

"The Second matrix is"
 1.  6.
 2.  4.
 3.  9.

"The product of the two matrices is"
 14.  41.
 32.  98.
```

Various type of plot

Introduction:

plot(x, y, 'style option')

plot(x, y, 'r')

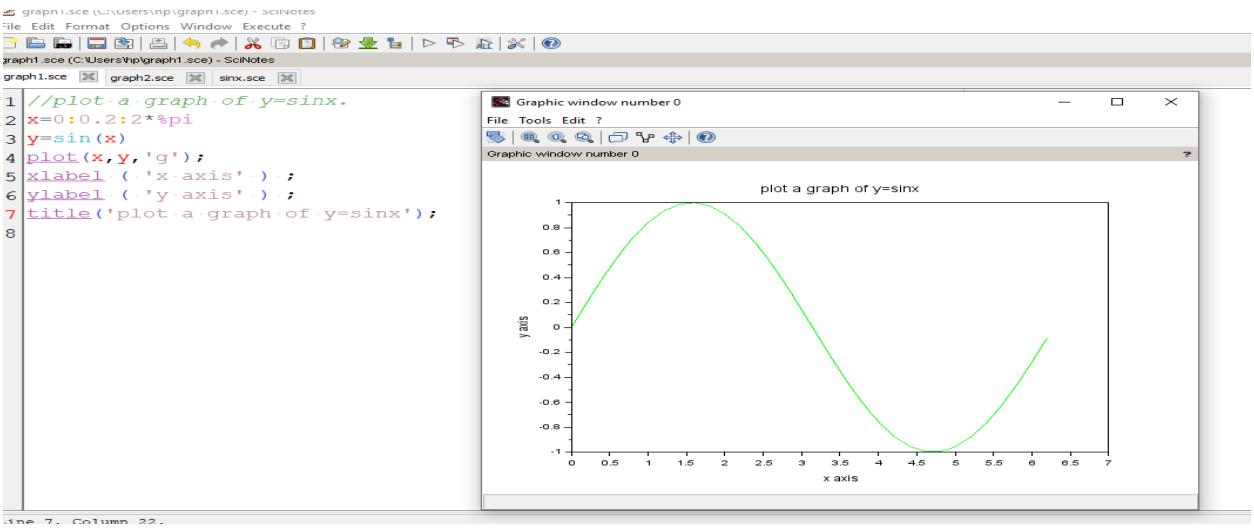
Line style and marker style of steeple options:

Color Style	**Line Style**	**Marker Style**
y= Yellow	– Solid	· Point
r= Red	: Dotted	o Circle
g= Green	–· Daggered	+ Pluse
b= Blue	– – Dashed	* Star
k= Black	C Cyan	S Square
	m Magenta	d Diamond
		v Triangle
		^ Triangle
		< Triangle
		> Triangle
		P Pentagram
		h Hexagram

<u>Practical 9</u>: Draw the graph of following functions for a given data.

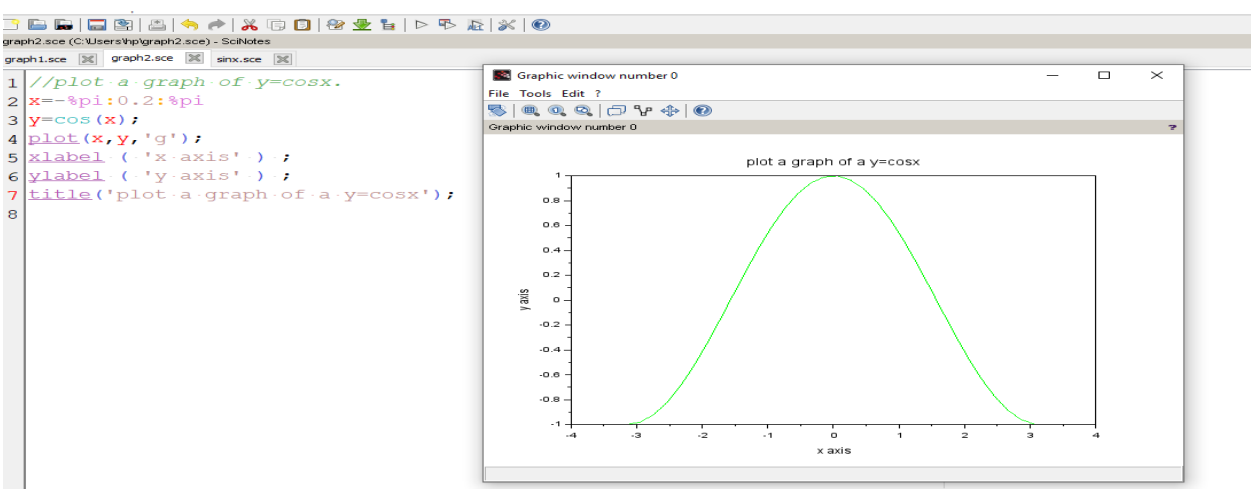
(1) $y = \sin \sin x$, $x \in [0, 2\pi]$

```
// Plot a graph of y=sinx.
x=0:0.2:2*%pi
y=sin(x);
plot(x,y,'g');
xlabel ( 'x axis' ) ;
ylabel ( 'y axis' ) ;
title('plot a graph of y=sinx');
```



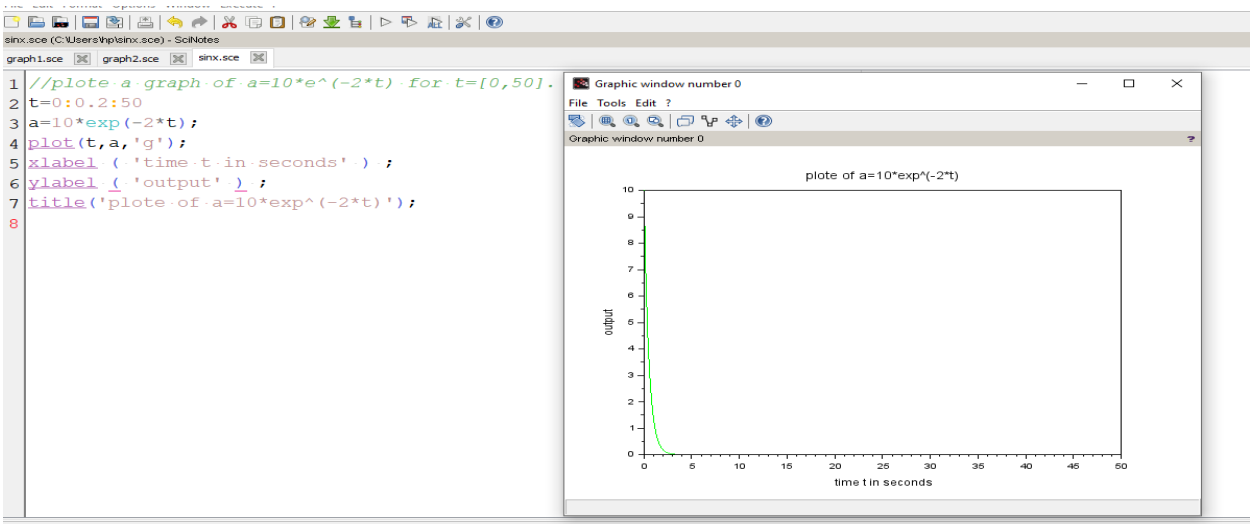
(2) $y = \cos \cos x$, $x \in [-\pi, \pi]$

```
// Plot a graph of y=cosx.
x=-%pi : 0.2: %pi
y=cos(x);
plot(x,y,'g');
xlabel ( 'x axis' ) ;
ylabel ( 'y axis' ) ;
title('plot a graph of a y=cosx');
```



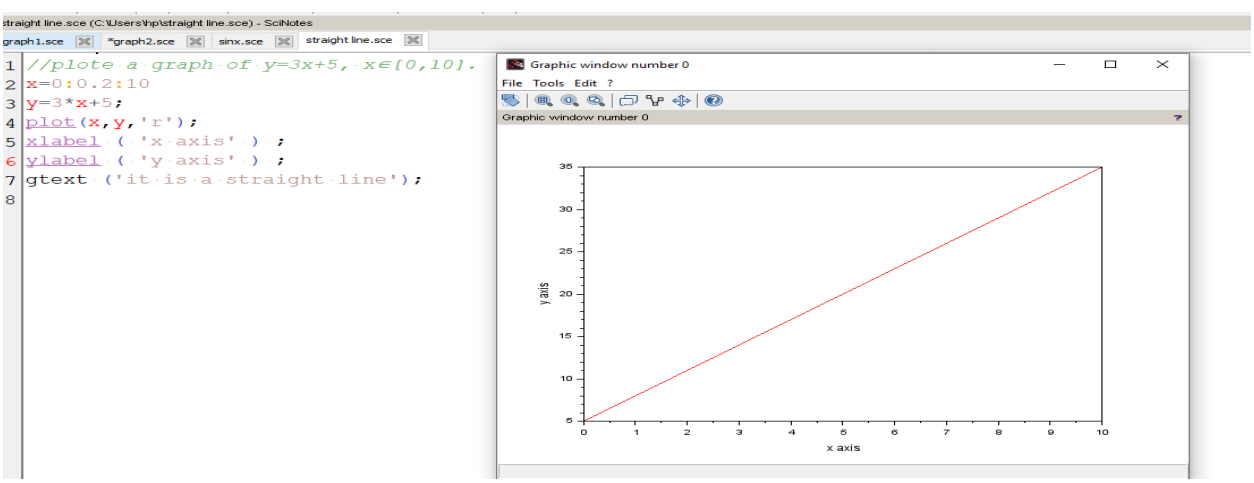
(3) $a = 10e^{-0.2t}, t \in [0, 50]$

```
// Plote a graph of a=10*e^(-2*t) for t=[0,50].
t=0:0.2:50
a=10*exp(-2*t);
plot(t, a, 'g');
xlabel ( 'time t in seconds' ) ;
ylabel ( 'output' ) ;
title('plote of a=10*exp^(-2*t)');
```



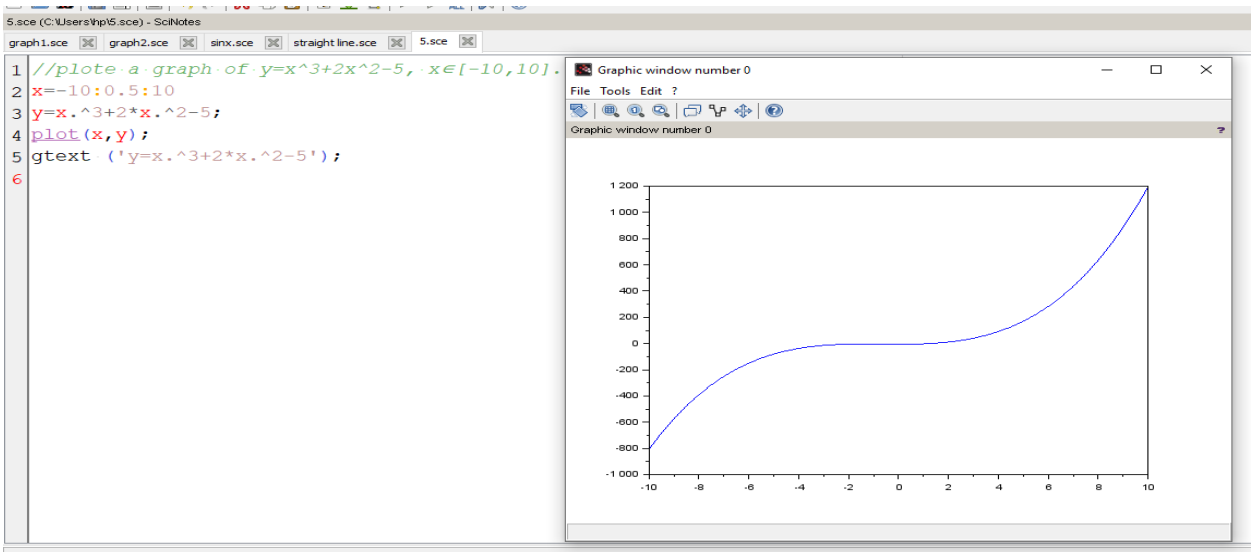
(4) $y = 3x + 5, x \in [0,10]$

```
// Plote a graph of y=3x+5, x∈[0,10].
x=0:0.2:10
y=3*x+5;
plot(x,y,'r');
xlabel ( 'x axis' ) ;
ylabel ( 'y axis' ) ;
gtext ('it is a straight line');
```



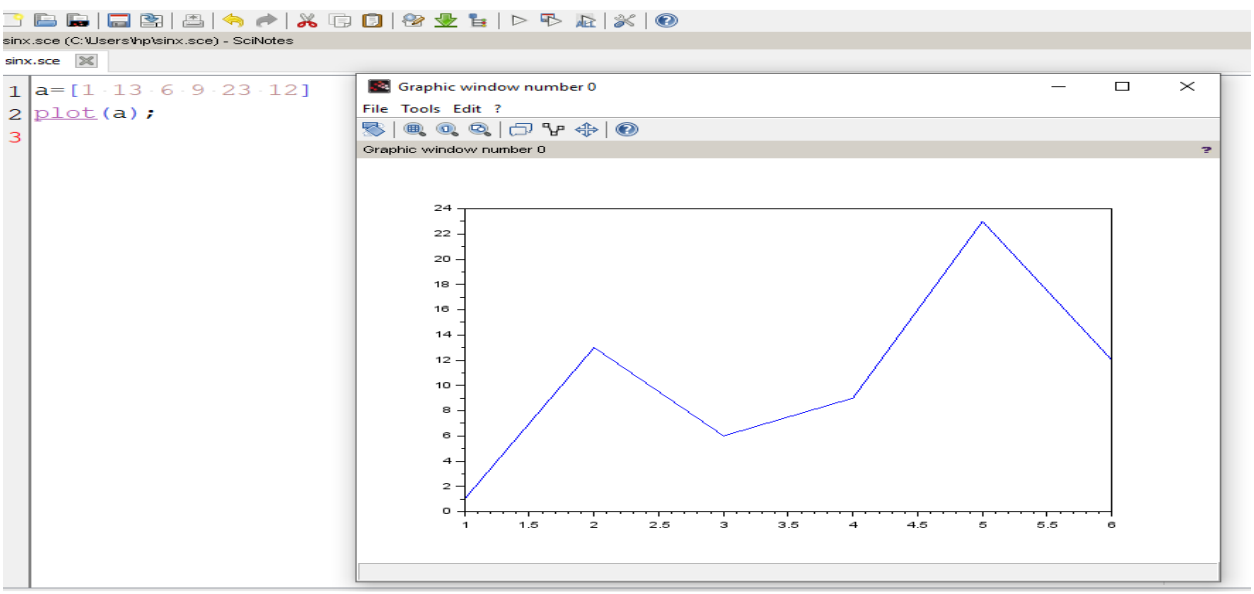
(5) $y = x^3 + 2x^2 - 5, x \in [-10,10]$

```
//plote a graph of y=x^3+2x^2-5, x∈[-10,10].
x=-10:0.5:10
y=x.^3+2*x.^2-5;
plot(x,y);
gtext ('y=x.^3+2*x.^2-5');
```



(6) $a = [1 \quad 13 \quad 6 \quad 9 \quad 23 \quad 12]$

```
// plot a graph for a= a=[1   13   6   9   23   12]/
a=[1 13 6 9 23 12]
plot(a);
```



<u>Practical 10</u>: Write a SCILAB program to draw a multiple graph using style operation, hold command, line command.

Style operation:

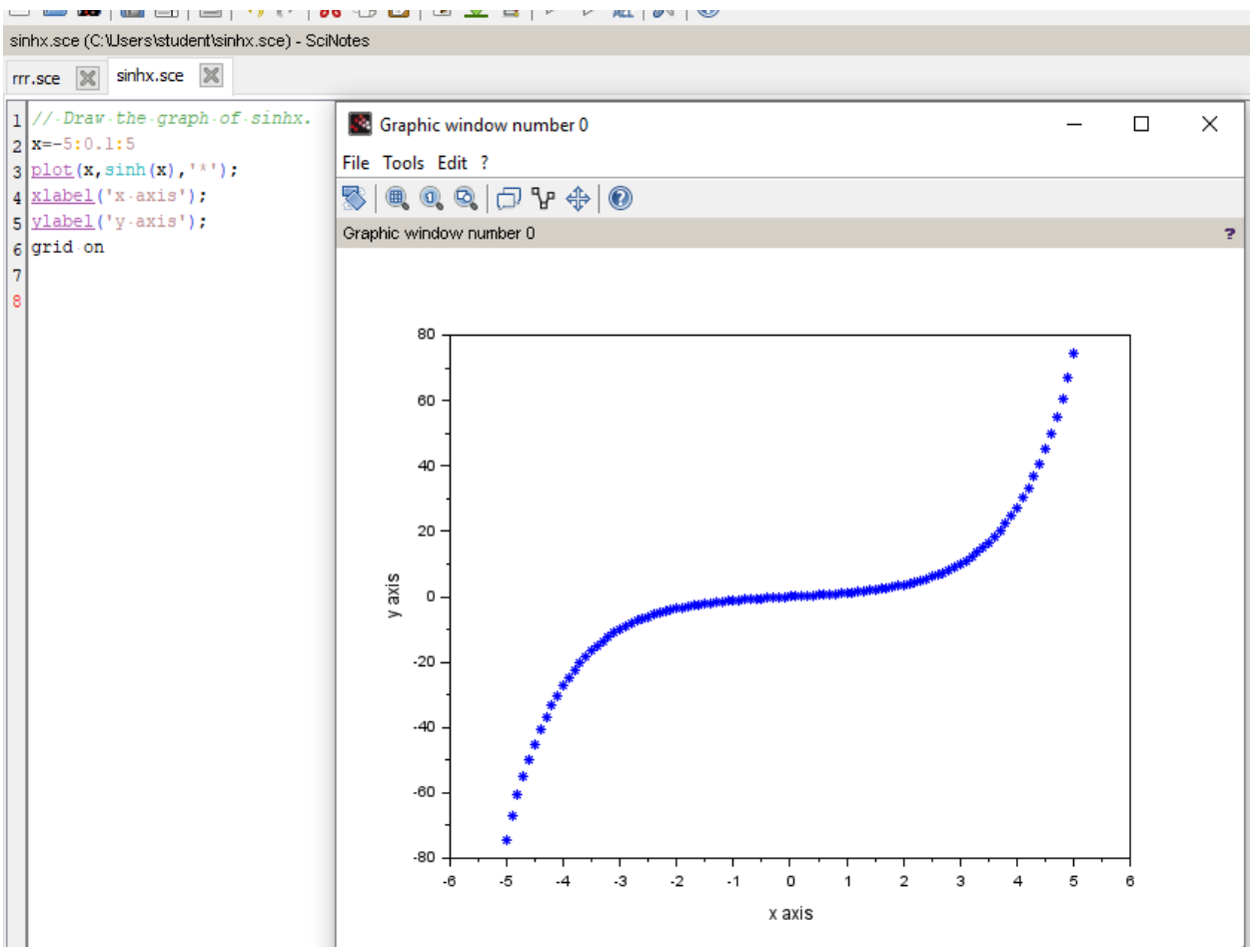
plot(x, y, 'style soluution')

plot(x, y, 'r')

we can use line style and marker style.

(1) Draw the graph of $y = \sin x$ **using style operation.**

```
// Draw the graph of sinhx.
x = -5:0.1:5
plot(x,sinh(x),'*');
xlabel('x axis');
ylabel('y axis');
grid on
```

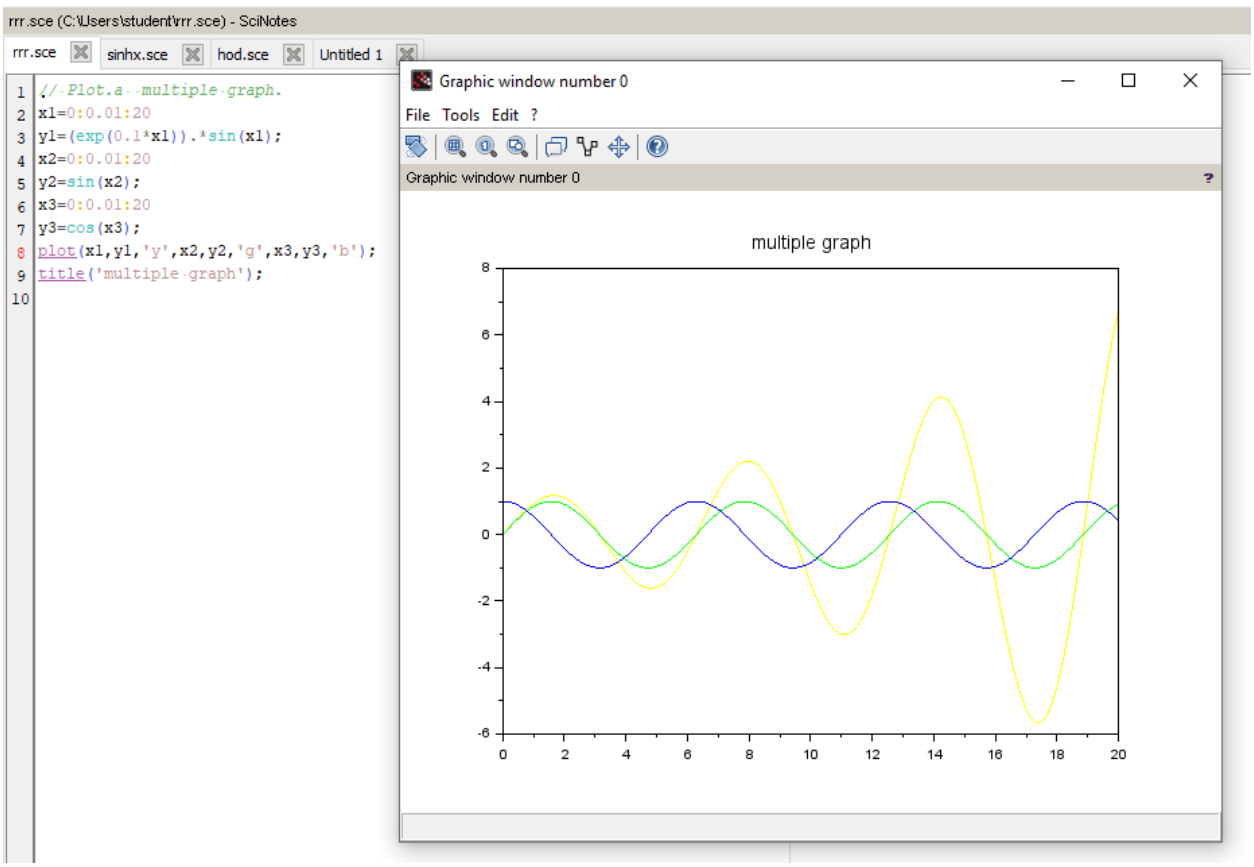


(2) Write a scilab program to draw the multiple curves for following functions

$y = e^{0.1x}\sin x;\ y = \sin x$ and $y = \cos x$.

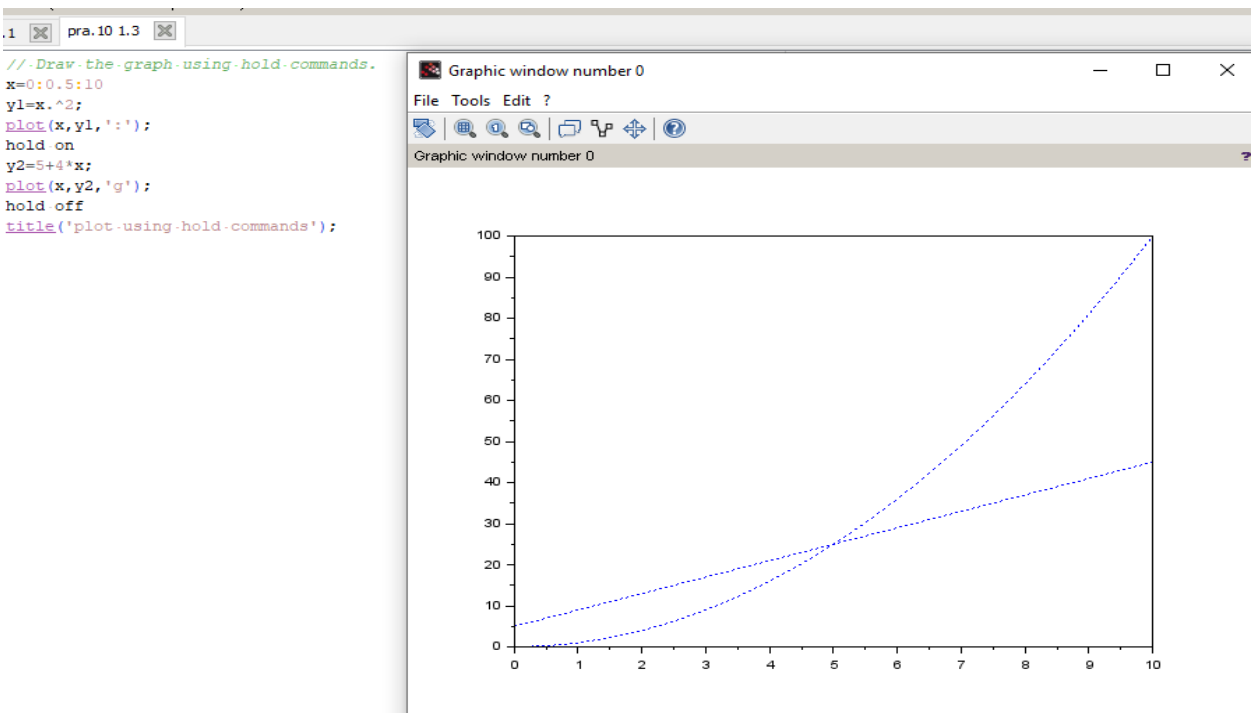
Note: we can generate multiple graphs using plot command.

```
// Plot.a  multiple graph.
x1=0:0.01:20
y1=(exp(0.1*x1)).*sin(x1);
x2=0:0.01:20
y2=sin(x2);
x3=0:0.01:20
y3=cos(x3);
plot(x1,y1,'y',x2,y2,'g',x3,y3,'b');
title('multiple graph');
```



(3) Write a SCILAB program to draw curves for $y = 5 + 4x$ and $y = x^2$ using hold on and hold off commands.

```
// Draw the graph using hold commands.
x=0:0.5:10
y1=5+4*x;
plot(x,y1,':');
hold on
y2=x.^2;
plot(x,y2,'g');
hold off
title('plot using hold commands');
```



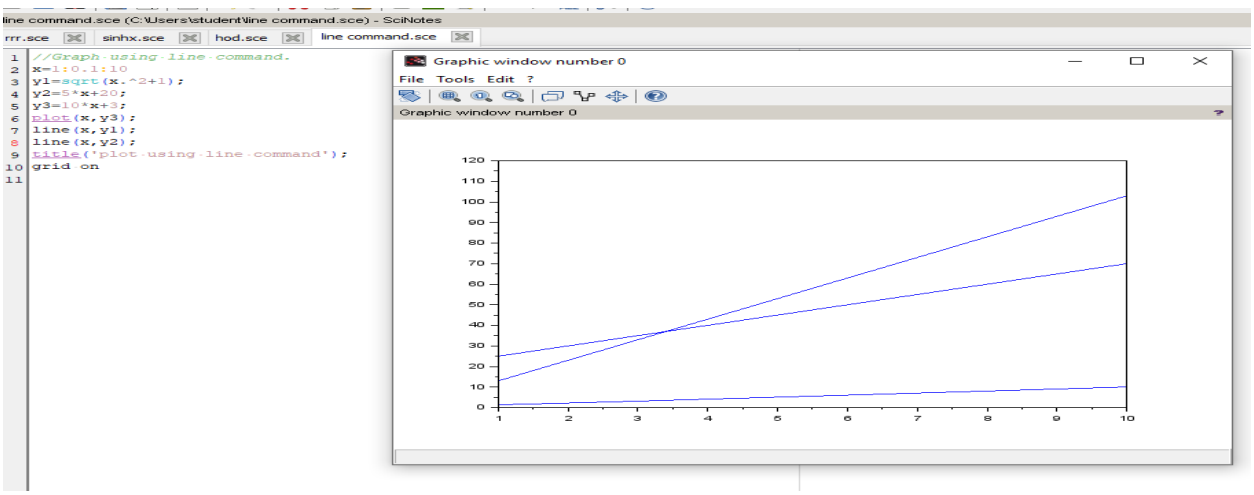
Line Command: once a graph is generated in a graphic window more plot can be added using the 'Line command'.

The syntax for line command is

line(x data, y data, parameter, name, parameter, value)

(4) Write a SCILAB program to draw curves $y = 5x + 20$ and $y = 10x + 3; x \in [1,10]$.

```
//Graph using line command.
x=1:0.1:10
y1=sqrt(x.^2+1);
y2=5*x+20;
y3=10*x+3;
plot(x,y3);
line(x,y1);
line(x,y2);
title('plot using line command');
grid on
```



Subplot: the graphic window can be subdivided into many subplots using subplot command.

Syntax:

subplot(m,n,p) **or** subplot(mnp)

Arguments:

m, n, p

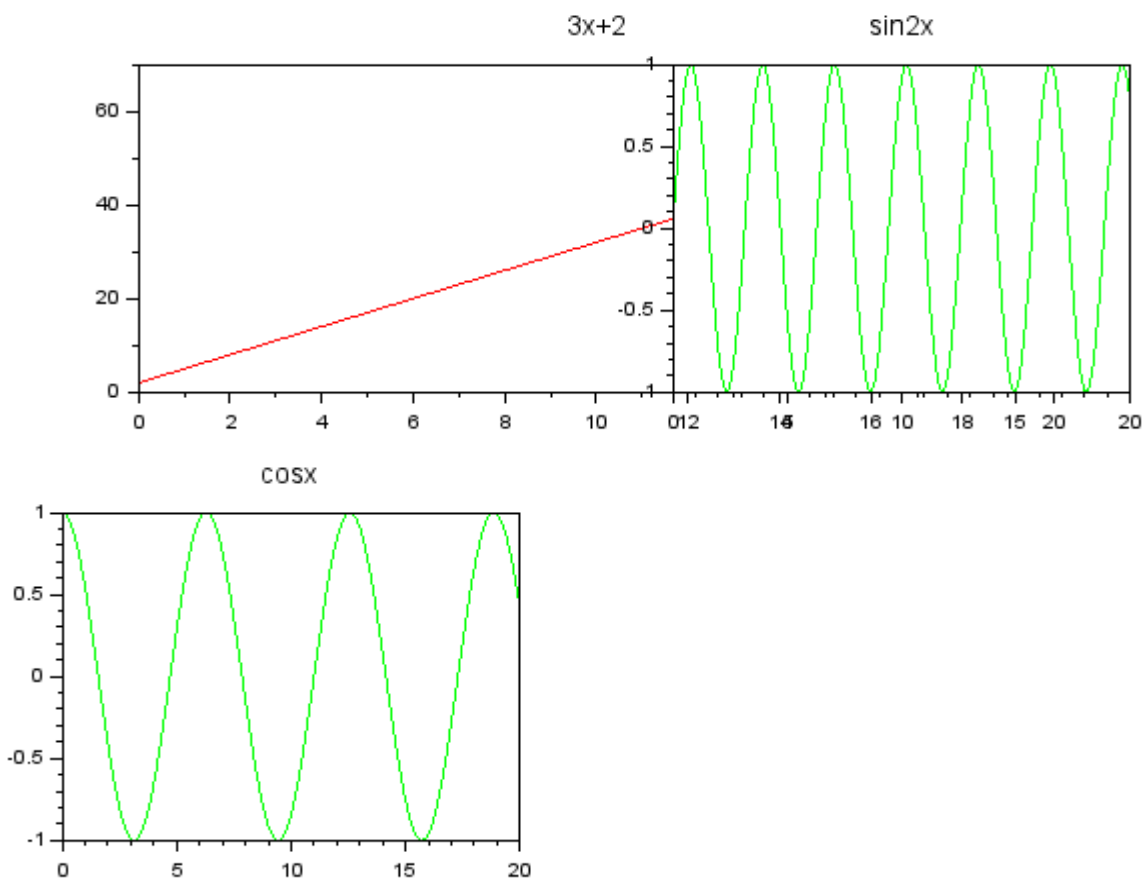
integers > 0. (m,n) = numbers of rows and columns of the virtual grid. p = index of the targeted cell.

mnp

an integer with decimal notation mnp

<u>Practical 11</u>: Write a SCILAB program to draw a graph using subplot command.

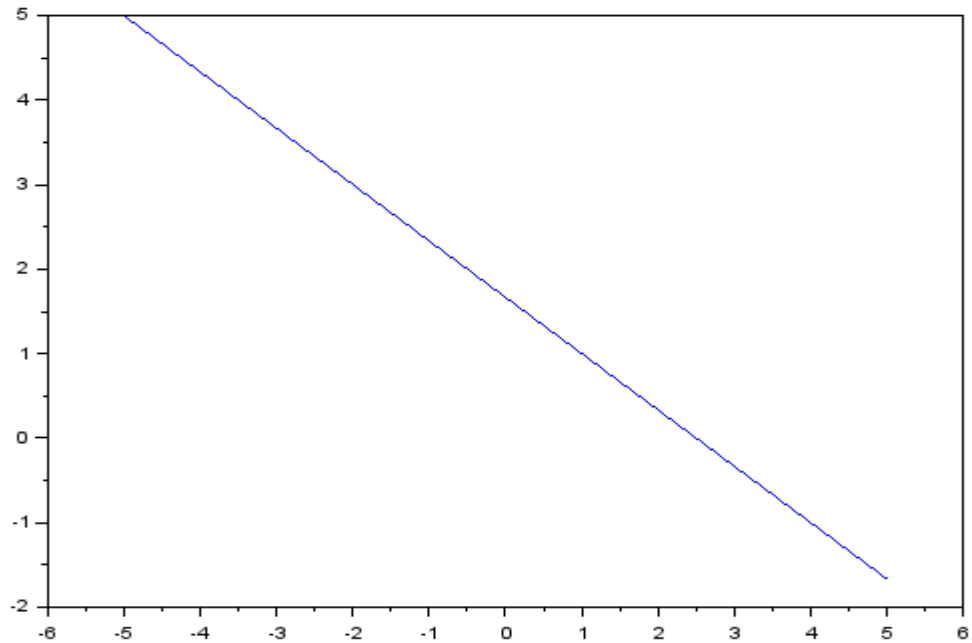
```
// Plot using subplot command.
x=0:0.1:20
y1=3*x+2;
subplot(2,1,1);
plot(x,y1,'r');
title('3x+2');

y2=sin(2*x);
subplot(2,2,2);
plot(x,y2,'g');
title('sin2x');

y3=cos(x);
subplot(2,2,3);
plot(x,y3,'g');
title('cosx');
```



<u>Practical 12</u>: Write a SCILAB program to draw Simple 2-D graphs.

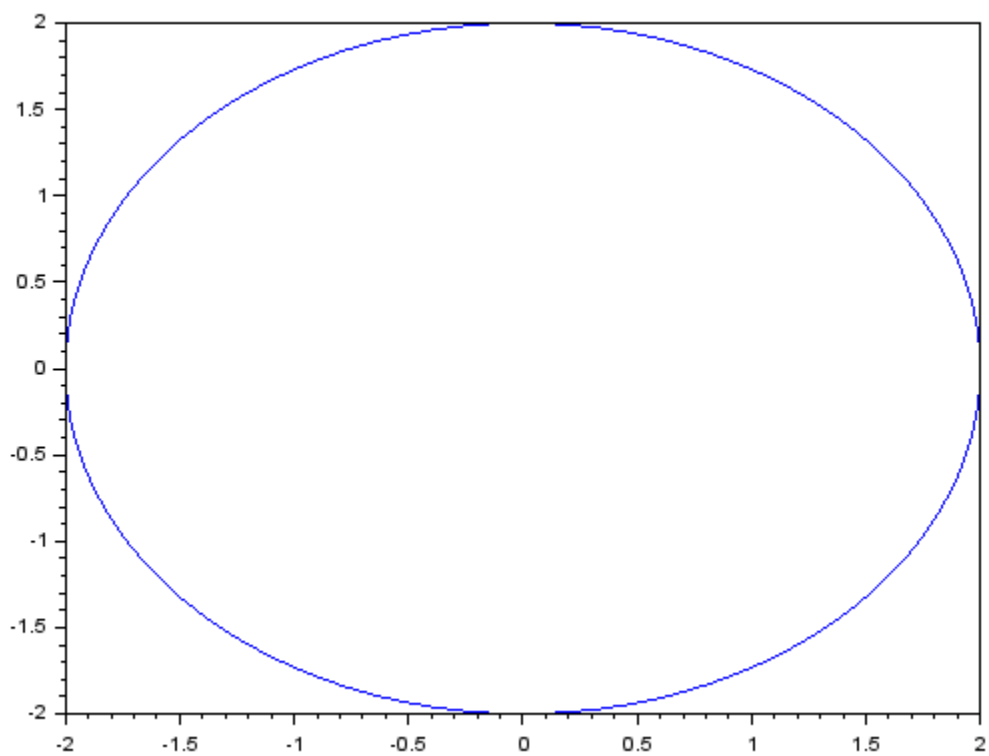
(1) Draw the graph of a line $2x + 3y = 0, \quad x \in [-5,5]$.

```
// Graph of a line 2x+3y-5=0.
x=linspace(-5,5,10)
y=(-2/3)*x+(5/3);
plot(x,y)
```



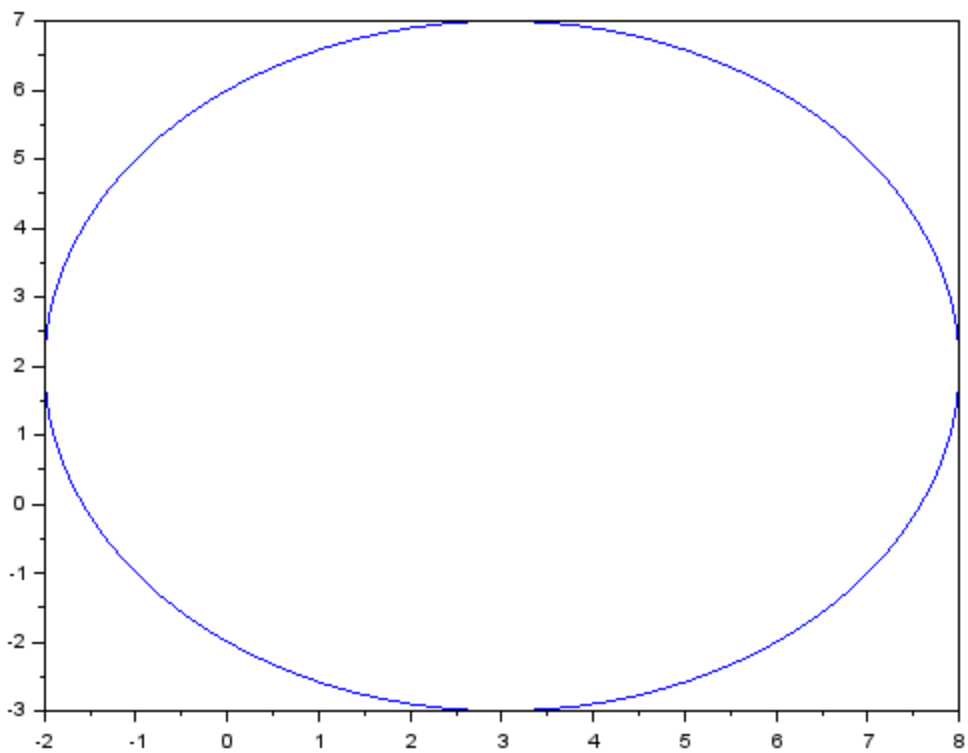
(2) Draw the graph of a circle $x^2 + y^2 = 4; x = r\cos\theta, y = r\sin\theta$**.**

```
//center of the circle
xc = 0;
yc = 0;

//radius of the circle
r = 2;

//dummy variable for angle in range [0, 2*%pi]
a = linspace(0, 2*%pi, 100);
//x axis
x = xc + r*cos(a);
//y axis
y = yc + r*sin(a);

//plot the circle
plot(x, y);
```



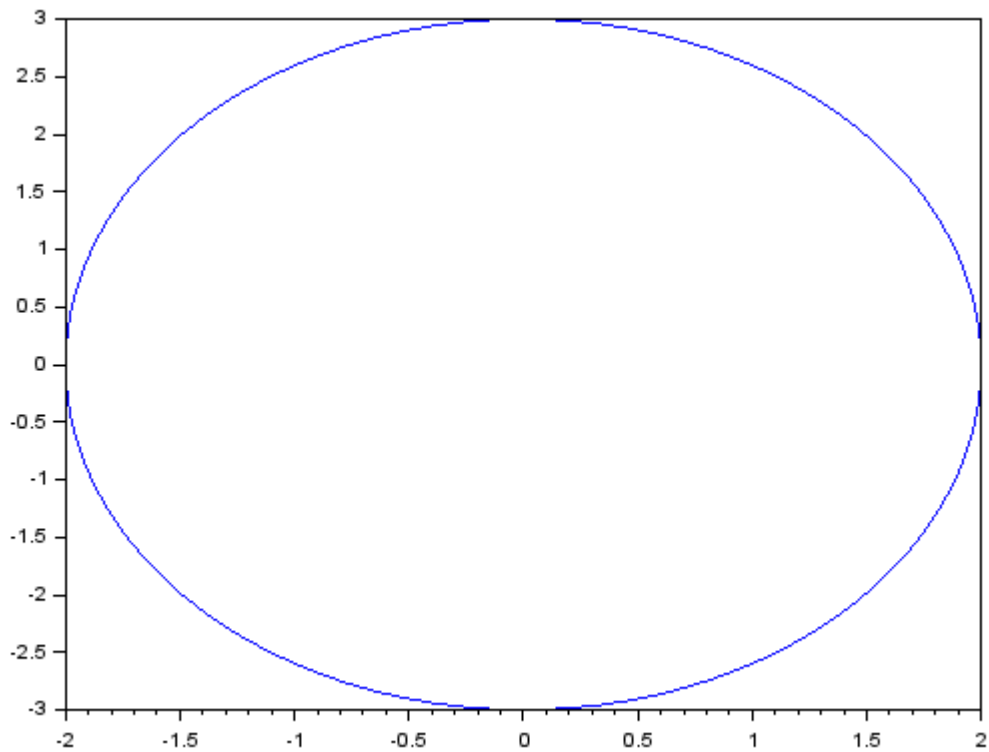
(3) Draw the graph of a circle $(x-3)^2 + (y-2)^2 = 5^2$.

```
//center of the circle
xc = 3;
yc = 2;

//radius of the circle
r = 5;

//dummy variable for angle in range [0, 2*%pi]
a = linspace(0, 2*%pi, 100);

//x axis
x = xc + r*cos(a);

//y axis
y = yc + r*sin(a);

//plot the circle
plot(x, y);
```



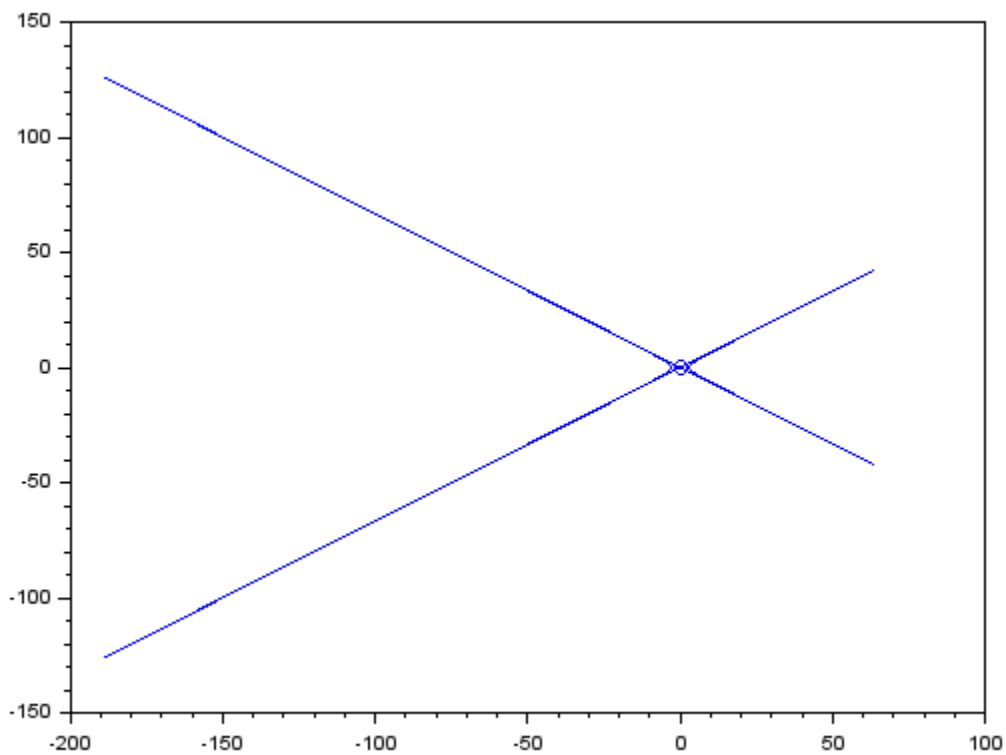
(4) Draw the graph of an ellipse $\frac{x^2}{4} + \frac{y^2}{9} = 1$.

```
//radius of the ellipse
r = 1;

//dummy variable for angle in range [0, 2*%pi]
a = linspace(0, 2*%pi, 100);

//x axis
x =2*cos(a);

//y axis
y =3*sin(a);

//plot the ellipse
plot(x, y);
```



(4) Draw the graph of a hyperbola $\frac{x^2}{9} - \frac{y^2}{4} = 1.$

```
//radius of the hyperbola
r = 1;

//dummy variable for angle in range [0, 2*%pi]
a = linspace(0, 2*%pi, 100);

//x axis
x =3*sec(a);;

//y axis
y =2*tan(a);

//plot the hyperbola
plot(x, y)
```

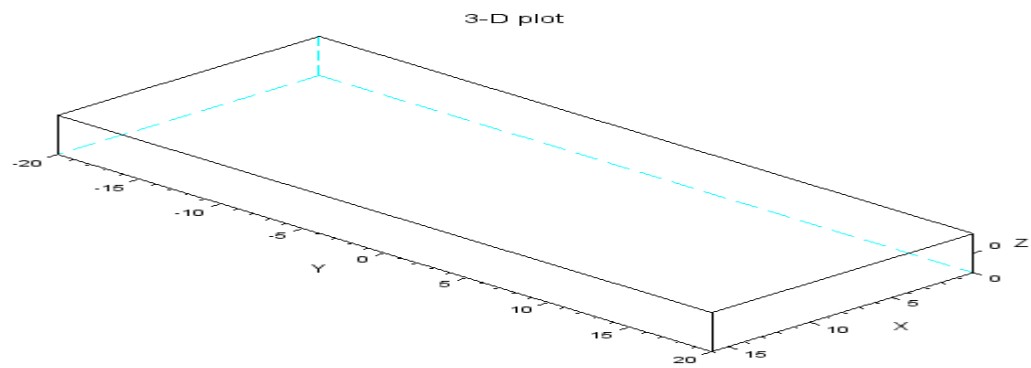


<u>Practical 13</u>: Write a SCILAB program to draw the 3-D plot such as plot-3, bar-3, mesh, surf, etc.

(1) Plot-3: plot3d(x, y, t, ‘style operation’)

Ex: $x = t^2, y = 4t$; for $-4 \leq t \leq 4$

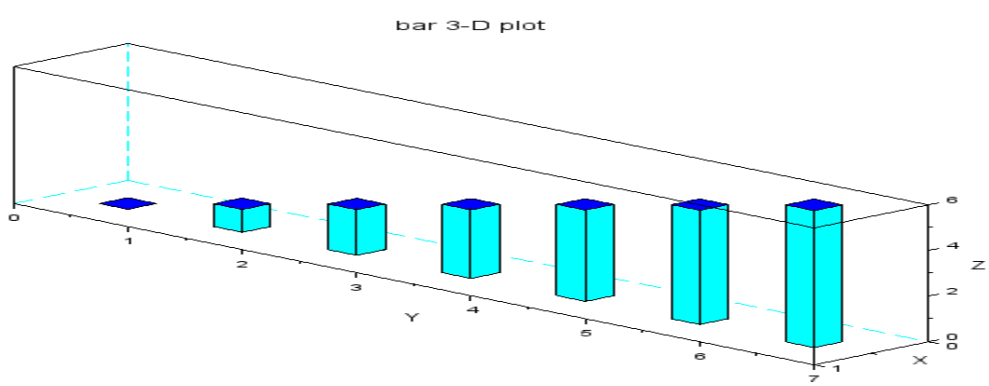
```
// Plot 3
t = - 4:0.5:4;
x = t.^2;
y = 4*t;
plot3d(x,y,t,'k');
title('3 d plot');
```



(2) bar-3: bar3d(x, y, ‘style operation’)

Ex: Draw the 3-D bar graph for given data x= [0 1 2 3 4 5 6], y = [5 17 25 21 31 27 19.]

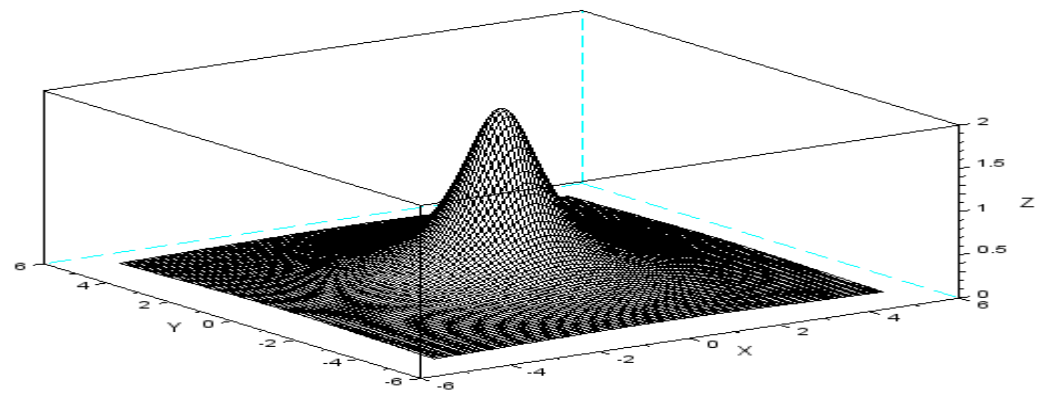
```
// Plot a bar graph.
x=[0 1 2 3 4 5 6];
y=[5 17 25 21 31 27 19];
bar3d(x,y,'r');
title('bar 3-D plot');
```



(3) Mesh function: mesh(x,y,z)

Ex: Draw the mesh plot for the equation $z = \frac{2}{(x^2+y^2+1)}$**;** $-5 \le y \le 5$**.**

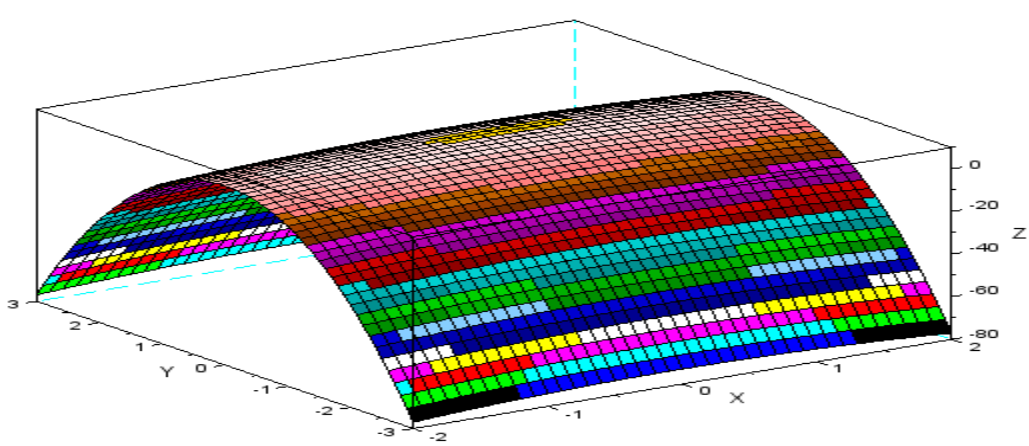
```
// Draw the mesh plot for the equation z=2/(x.^2+y.^2+1); -5 ≤ y ≤ 5.
[x,y]=meshgrid(-5:0.1:5);
z=2./(x.^2+y.^2+1);
mesh(x,y,z);
grid on
```



(4) Surf function: Surf(x,y,z)

EX: Draw the surf plot of $f(x, y) = 8 - x^2 - 9y^2;\ -2 \le x \le 2$ and $-3 \le y \le 3$.

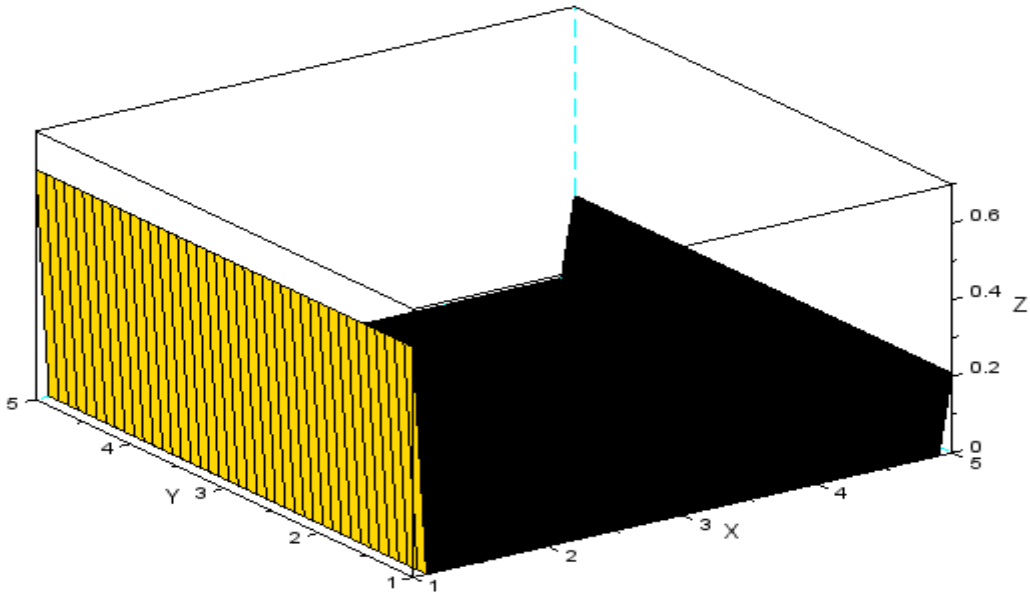
```
// plot using surf function.
[x,y]=meshgrid(-2:0.1:2,-3:0.1:3);
z=8-x.^2-9*y.^2;
surf(x,y,z);
grid on
```



(5) Draw the graph of $f(x,y) = z = \frac{5x}{x^2+y^2+1}$**.**

Input:

```
[x,y]=meshgrid(1:0.1:5);
z=5*x/(x.^2+y.^2+1);
surf(x,y,z);
grid on
```



<u>Practical 14</u>: Write a SCILAB program to check linear independence of vectors using conditional statement.

Input:

```
clc;
clear;
A = input('enter the vector:');
disp("Matrix of vectors (columns):");
disp(A);

[rows, cols] = size(A);
if rows < cols then
   disp("Warning: More vectors than dimensions — they cannot be independent.");
end

r = rank(A); // Computes rank of matrix

if r == rows then
   disp("The vectors are LINEARLY INDEPENDENT.");
else
   disp("The vectors are LINEARLY DEPENDENT.");
end

mprintf("Rank of matrix: %d\n", r);
mprintf("Number of vectors: %d\n", rows);
```

/***Output**:

```
enter the vector: [1 2 3;4 5 6;7 8 9]

 "Matrix of vectors (columns):"

  1.  2.  3.
  4.  5.  6.
  7.  8.  9.

 "The vectors are LINEARLY DEPENDENT."
Rank of matrix: 2
Number of vectors: 3
```

<u>Practical 15</u>: Write a SCILAB program to check orthogonality of vectors by inner product.

Input:

```
clc;clear;
v1 = input('enter fist vector:');
v2 = input('enter second vector:');

// Compute inner product
inner_product = v1 * v2';

// Display inner product
disp("Inner Product = ");
disp(inner_product);

// Check orthogonality
if inner_product == 0 then
   disp("The vectors are ORTHOGONAL.");
else
   disp("The vectors are NOT orthogonal.");
end
```

/***output:**

```
enter fist vector: [2 4 6]
enter second vector: [4 4 -4]
  "Inner Product = "
   0.
  "The vectors are ORTHOGONAL."

enter fist vector: [1 2 3]
enter second vector: [2 4 6]
  "Inner Product = "
   28.
  "The vectors are NOT orthogonal."
```

OR

```
clc;clear;
v1 = input("Enter first vector: ");
v2 = input("Enter second vector: ");

// Check if dimensions match
if length(v1) <> length(v2) then
   disp("Vectors must have the same dimension.");
else
   ip = v1 * v2';   // inner product

   disp("Inner Product = ");
   disp(ip);

   if ip == 0 then
      disp("Vectors are ORTHOGONAL.");
   else
      disp("Vectors are NOT orthogonal.");
   end
end


/*Output:

Enter first vector: [1 2 3]
Enter second vector: [2 5]
  "Vectors must have the same dimension."


Enter first vector: [ 1 2 3]
Enter second vector: [4 4 4]
  "Inner Product = "
   24.
  "Vectors are NOT orthogonal."

Enter first vector: [2 6 4]
Enter second vector: [6 4 -9]
  "Inner Product = "
   0.
  "Vectors are ORTHOGONAL."
```

Signature of Faculty:

Date: